ECE 414/517: Reinforcement Learning

Homework 3 + Coding Assignment 3

Fall 2026

Instructor: Fei Liu

Released: Tuesday, November 10, 2026
Due: Monday, November 30, 2026 at 11:59 p.m. ET

Purpose and learning objectives

This assignment covers Value-function Approximation I/II and Policy Gradients I/II. You will (i) compute gradient Monte Carlo, semi-gradient TD, and semi-gradient SARSA updates; (ii) derive and apply a softmax policy gradient; (iii) explain and implement value baselines; and (iv) compare REINFORCE and on-policy actor–critic in LunarLander. The coding task uses small, supplied neural networks; your work focuses on the learning rules, not network architecture. PPO and DQN are not part of this assignment.

Submission instructions

Submit the following two items separately on Canvas.

1. **Homework 3 (written): submit a single PDF only.** Include answers to Problems 1–5 with intermediate calculations and reasoning. Do not put this written PDF in the coding ZIP.

2. **Coding Assignment 3: submit a single ZIP containing everything.** Put your coding report `report.pdf`, completed `coding/`, and generated `results/` and `figures/` together in this ZIP. Do not submit the coding report PDF separately. Include source CSVs, configuration JSONs, and the final `agent.pt` checkpoint in each submitted run folder. No intermediate checkpoints are needed. Exclude virtual environments and caches. ECE 517 students also complete E1.

Follow the course collaboration, late-work, and academic-integrity policies on Canvas. The written and coding core portions are each worth 100 points for *both* courses. ECE 517 has an additional required 10-point coding extension: 210 required points total, compared with 200 for ECE 414. There are no bonus tasks.

Homework 3: Written Problems (100 points)

Conventions

Reward R_{t+1} follows action A_t; terminal value is zero. Feature vectors and parameter vectors are columns. Reset to the stated initial parameters for independent subparts; do not carry updates between questions. Round numerical answers to four decimal places. A squared-error objective with factor $1/2$ and one without that factor have different gradients; use the exact objective specified in each question.

1 Linear value approximation and gradient Monte Carlo (20 points)

Use $\hat{v}_{\mathbf{w}}(s) = \mathbf{w}^\top \mathbf{x}(s)$ with

$$\mathbf{x}(A) = (1, 0)^\top, \quad \mathbf{x}(B) = (1, 1)^\top, \quad \mathbf{x}(C) = (1, 2)^\top, \qquad \mathbf{w} = (1, 0.5)^\top.$$

A fixed policy produces $A \xrightarrow{1} B \xrightarrow{-1} C \xrightarrow{3} T$, where T is terminal. Use $\gamma = 0.9$ and $\alpha = 0.1$.

(a) Using the initial weights $\mathbf{w} = (1, 0.5)^\top$, compute the estimated state values $\hat{v}_{\mathbf{w}}(A)$, $\hat{v}_{\mathbf{w}}(B)$, and $\hat{v}_{\mathbf{w}}(C)$. Also compute the sampled returns G_0, G_1, G_2 from the given episode. [5 pts]

(b) For one sample (s, G), derive the gradient of $L(\mathbf{w}) = \frac{1}{2}(G - \mathbf{w}^\top \mathbf{x}(s))^2$ and the gradient-descent update. State which quantities are held fixed. [5 pts]

(c) Update $\mathbf{w}$ using only the sample at B. Show the prediction error, gradient, new weights, and new predictions at A, B, C. [6 pts]

(d) All three state-value estimates use the *same* two weights: $\hat{v}_{\mathbf{w}}(A) = w_1$, $\hat{v}_{\mathbf{w}}(B) = w_1 + w_2$, and $\hat{v}_{\mathbf{w}}(C) = w_1 + 2w_2$. Using your results from part (c), compute the change in each estimate (new value minus initial value). Explain why the estimate at A changes even though the update uses only the sample at B and A occurs before B in the episode. Explain how changing the shared weights affects the three value estimates. No additional updates are performed at A or C. [4 pts]

2 TD and SARSA with Linear Value Approximation (20 points)

Background and Update Equations. In this problem, we consider Temporal-Difference (TD) learning and SARSA with linear function approximation, where state-value and action-value functions are represented by parameterized models rather than tabular entries. At each step, the current value estimates are used to compute the TD target y and TD error δ, which are then used to update the model weights. All quantities on the right-hand side of the update equations are evaluated using the weights before the update.

For state-value prediction, $\hat{v}_{\mathbf{w}}(s) = \mathbf{w}^\top \mathbf{x}(s)$. For a nonterminal transition (s, r, s'), use

$$y_{\text{TD}} = r + \gamma \hat{v}_{\mathbf{w}}(s'), \qquad \delta_{\text{TD}} = y_{\text{TD}} - \hat{v}_{\mathbf{w}}(s), \qquad \mathbf{w}^+ = \mathbf{w} + \alpha \delta_{\text{TD}} \mathbf{x}(s).$$

For action-value learning, $\hat{q}_{\mathbf{u}}(s, a) = \mathbf{u}^\top \mathbf{x}(s, a)$. SARSA uses the *observed next action* a' in the transition (s, a, r, s', a'). For a nonterminal s', use

$$y_{\text{SARSA}} = r + \gamma \hat{q}_{\mathbf{u}}(s', a'), \qquad \delta_{\text{SARSA}} = y_{\text{SARSA}} - \hat{q}_{\mathbf{u}}(s, a),$$

$$\mathbf{u}^+ = \mathbf{u} + \alpha\delta_{\text{SARSA}}\mathbf{x}(s,a).$$

The current feature vector determines which weights change; the next feature vector is used to calculate the target. Do not maximize over next actions. If s' is terminal, use $y = r$ in either update. These rules treat y as a constant when differentiating the sample loss $L = \frac{1}{2}(y - \hat{v}_{\mathbf{w}}(s))^2$ or $L = \frac{1}{2}(y - \hat{q}_{\mathbf{u}}(s,a))^2$. For example, with y held fixed,

$$\nabla_{\mathbf{u}} L = -\delta_{\text{SARSA}}\mathbf{x}(s,a), \qquad \mathbf{u}^+ = \mathbf{u} - \alpha\nabla_{\mathbf{u}} L.$$

The term *semi-gradient* refers to ignoring the dependence of the bootstrapped target on the weights during this differentiation.

(a) Use the initial features and weights from Problem 1(a), $\mathbf{w} = (1, 0.5)^\top$, with $\gamma = 0.9$ and $\alpha = 0.1$. Do not use the updated weights from Problem 1(c). For the transition $B \xrightarrow{-1} C$, compute the TD target $y = -1 + \gamma\hat{v}_{\mathbf{w}}(C)$, TD error, and update $\mathbf{w}^+ = \mathbf{w} + \alpha(y - \hat{v}_{\mathbf{w}}(B))\mathbf{x}(B)$. Recompute the prediction at B after the update. [6 pts]

(b) For a separate control problem, let $\hat{q}_{\mathbf{u}}(s,a) = \mathbf{u}^\top\mathbf{x}(s,a)$, $\mathbf{u} = (0.5, 1, -0.5)^\top$, $\mathbf{x}(s,a) = (1,2,0)^\top$, and $\mathbf{x}(s',a') = (1,0,1)^\top$. The observed transition is $(s,a,2,s',a')$ and s' is nonterminal. With $\gamma = 0.9$ and $\alpha = 0.1$, first calculate $\hat{q}_{\mathbf{u}}(s,a)$ and $\hat{q}_{\mathbf{u}}(s',a')$. Then use the provided SARSA equations to calculate the target, error, and updated weights. [6 pts]

(c) Using the definition above, explain what is held fixed in a *semi-gradient* update. For part (a), write the gradient of $\frac{1}{2}[y(\mathbf{w}) - \hat{v}_{\mathbf{w}}(B)]^2$ if the target's dependence on $\mathbf{w}$ were also differentiated, and identify the extra term. *Hint:* write $e(\mathbf{w}) = y(\mathbf{w}) - \hat{v}_{\mathbf{w}}(B)$, apply $\nabla_{\mathbf{w}}(\frac{1}{2}e^2) = e\nabla_{\mathbf{w}}e$, and use $\nabla_{\mathbf{w}}\hat{v}_{\mathbf{w}}(s) = \mathbf{x}(s)$. No stability or convergence argument is required. [4 pts]

(d) Consider the original trajectory $B \xrightarrow{-1} C \xrightarrow{3} T$, where T is terminal. Using $\gamma = 0.9$ and the initial value estimate $\hat{v}_{\mathbf{w}}(C)$ from Problem 1(a), compute and compare the one-step TD target and the Monte Carlo return from B. Which needs the rest of the episode? Which uses an estimated future value? [4 pts]

3 Softmax policies and REINFORCE (20 points)

At a particular state, there are two actions L, R. Their independent scalar logits are θ_L, θ_R, with $\pi_\theta(a \mid s) = e^{\theta_a}/(e^{\theta_L} + e^{\theta_R})$. Initially $\theta = (\log 3, 0)^\top$. The sampled action is R and its return is $G = 2$.

(a) Compute both action probabilities. Derive $\partial \log \pi_\theta(a \mid s)/\partial\theta_j = \mathbf{1}\{j = a\} - \pi_\theta(j \mid s)$. [8 pts]

(b) At episode time $t = 0$, use $\alpha_\theta = 0.1$ to perform $\theta^+ = \theta + \alpha_\theta G\nabla_\theta \log \pi_\theta(R \mid s)$. Compute both new logits and the new probability of R. [8 pts]

(c) Explain why policy sampling does not need an ϵ-greedy rule. Why should the loss use the log-probability of the *sampled* action? [4 pts]

4 Learned baselines and variance reduction (20 points)

(a) For a fixed state s, prove $\mathbb{E}_{A\sim\pi_\theta(\cdot|s)}[b(s)\nabla_\theta \log \pi_\theta(A \mid s)] = 0$ when the baseline does not depend on the sampled action. Start with the finite sum over actions and use $\sum_a \pi_\theta(a \mid s) = 1$. In the actor update, the baseline is treated as fixed. [8 pts]

(b) Use a learned linear baseline $b_{\mathbf{w}}(s) = \hat{v}_{\mathbf{w}}(s) = \mathbf{w}^\top \mathbf{x}(s)$, which predicts the return from state s. The baseline is this predicted value, not the parameter vector $\mathbf{w}$ itself. A batch contains two states with feature vectors $\mathbf{x}_1 = (1, 0)^\top$ and $\mathbf{x}_2 = (1, 1)^\top$, and corresponding sampled returns $G_1 = 3$ and $G_2 = 1$. Using the initial parameters $\mathbf{w} = (1, 1)^\top$, calculate both baseline predictions $b_{\mathbf{w}}(s_i)$, both differences between the sampled return and the baseline prediction $G_i - b_{\mathbf{w}}(s_i)$, and the loss $L_v(\mathbf{w}) = \frac{1}{2N} \sum_{i=1}^{N} (b_{\mathbf{w}}(s_i) - G_i)^2$, where $N = 2$. Then take one gradient-descent step on this loss to update $\mathbf{w}$, using $\alpha_v = 0.1$. [12 pts]

5 One-step actor–critic (20 points)

Remark: This problem is a simplified version of the learning update in the coding assignment. You are welcome to work on both simultaneously to connect the hand calculations with your implementation and check the actor and critic targets.

At time $t = 0$, a two-action softmax actor has logits $(\log 3, 0)^\top$ and selects R. The transition gives reward 1 and reaches nonterminal s'. The critic is $\hat{v}_{\mathbf{w}}(s) = \mathbf{w}^\top \mathbf{x}(s)$, where $\mathbf{w} = (2, 1)^\top$, $\mathbf{x}(s) = (1, 0)^\top$, and $\mathbf{x}(s') = (1, 1)^\top$. Use $\gamma = 0.9$, actor step size 0.1, and critic step size 0.2.

(a) Compute both initial critic predictions, the one-step target y, and the TD advantage $\delta = y - \hat{v}_{\mathbf{w}}(s)$. [4 pts]

(b) **Actor-Step:** Update the actor using $\theta^+ = \theta + 0.1\delta \nabla_\theta \log \pi_\theta(R \mid s)$. Calculate both new logits and the new probability of R. [5 pts]

(c) **Critic-Step:** Using the *same pre-update target*, update the critic by minimizing $\frac{1}{2}(y - \hat{v}_{\mathbf{w}}(s))^2$. Show its gradient and new weights. [4 pts]

(d) Start again from the initial parameters, but suppose the transition is terminal with reward -2. Recompute the target, advantage, and both updates. [4 pts]

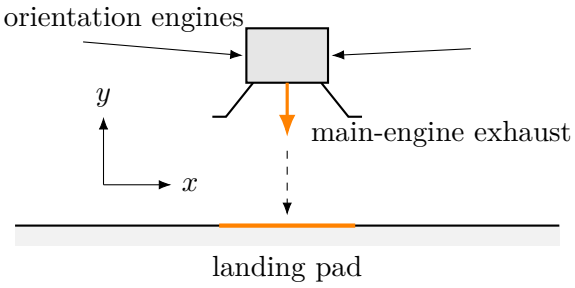
(e) Compare the targets used to train the value estimate in two methods: a REINFORCE baseline uses the complete sampled return $G_t = R_{t+1} + \gamma R_{t+2} + \cdots$, whereas a one-step actor–critic uses $y_t = R_{t+1} + \gamma \hat{v}_{\mathbf{w}}(S_{t+1})$ for a nonterminal transition. Answer each question briefly: (i) Which method must wait until the episode ends to obtain its target? (ii) Which method uses an estimated next-state value instead of the remaining sampled rewards? (iii) For that method, how can an inaccurate next-state value affect the target? [3 pts]

Coding Assignment 3: Learning to Land (100 points)

LunarLander task

Control a lunar lander so that it approaches the landing pad and lands safely. We use Gymnasium's `LunarLander-v3`, discrete actions, default gravity, and wind disabled. It uses Box2D, not MuJoCo. The environment provides eight observations: horizontal/vertical position, horizontal/vertical velocity, angle, angular velocity, and two leg-contact indicators. These are the environment's scaled observations; do not reinterpret all entries as SI measurements.

There are four actions: 0 does nothing, 1 fires the left orientation engine, 2 fires the main engine, and 3 fires the right orientation engine. The reward accounts for approach, motion, orientation, fuel use, contact, and the final outcome. Use the original rewards without adding penalties or shaping. Official environment documentation describes the full reward and observation conventions.



Task horizon. This assignment defines an episode as a finite task of at most $H = 1000$ actions. The supplied program appends remaining time $(H - t)/H$ to the eight observations, so both networks receive nine inputs. At physical termination *or* at this intrinsic horizon, the future value is zero. The final reward is still included. Thus `ended` is `terminated or truncated` here. This is a deliberate finite-horizon convention, not a general rule for all Gymnasium time limits. An external cutoff of a continuing task would require different handling.

Algorithms and loss conventions

Let B be the number of complete episodes in a training batch and N their total number of transitions. Use $\gamma = 0.99$. For the discounted finite-horizon objective $J(\theta) = \mathbb{E}[\sum_{t=0}^{T-1} \gamma^t R_{t+1}]$, the actor loss is

$$L_\pi(\theta) = -\frac{1}{B} \sum_{i=1}^{B} \sum_{t=0}^{T_i-1} \gamma^t \log \pi_\theta(A_{i,t} \mid S_{i,t}) \operatorname{stopgrad}(\hat{A}_{i,t}).$$

Here `stopgrad`, implemented by `.detach()`, treats a quantity as constant while differentiating. The program supplies γ^t, resetting $t = 0$ at every episode. Do not multiply by that factor a second time inside the return or advantage. Here $\hat{A}_t$ simply means the number multiplying the sampled action's log-probability in the actor loss; no separate advantage-function theory is required. Existing code arguments and function names call this number `advantages`. Keep those names unchanged. For REINFORCE it is the return itself; with a baseline it is return minus predicted value; for actor–critic it is the TD target minus predicted value. The three required methods differ in the actor update weight and value target:

Method	Actor update weight $\hat{A}_t$	Value target y_t
M1: REINFORCE	G_t	No value update
M2: REINFORCE + baseline	$G_t - \hat{v}(S_t)$	G_t
M3: One-step actor–critic	$y_t - \hat{v}(S_t)$	$R_{t+1} + \gamma(1 - d_t)\hat{v}(S_{t+1})$

$d_t = 1$ at either task ending and zero otherwise. Returns satisfy $G_t = R_{t+1} + \gamma G_{t+1}$, with $G_T = 0$. The value loss is $L_v = \frac{1}{N}\sum_{j=1}^{N}(\hat{v}(S_j) - \text{stopgrad}(y_j))^2$, *without* a factor 1/2. Unlike the numerical written updates, the supplied program uses Adam. Do not implement manual parameter changes in your loss functions.

How the table connects to your functions. The middle column supplies the weight used by `policy_loss` to train the actor; the last column supplies the target used by `value_loss` to train the value network.

- **M1: REINFORCE.** `returns_to_go` produces G_t. The supplied program passes G_t directly as the update weight to `policy_loss`. There is no baseline subtraction or value-network update.
- **M2: REINFORCE + baseline.** `returns_to_go` produces G_t. For the actor, `baseline_advantages` combines G_t with $\hat{v}(S_t)$ to produce the middle-column weight $G_t - \hat{v}(S_t)$, which is passed to `policy_loss`. Separately, `value_loss` compares the value prediction with the last-column target G_t. Thus the baseline is trained to predict the return, not the return-minus-baseline.
- **M3: One-step actor–critic.** `td_targets` first produces the last-column target y_t using the reward and next-state prediction. For the actor, `td_advantages` subtracts the current-state prediction from y_t to produce the middle-column weight $y_t - \hat{v}(S_t)$, which is passed to `policy_loss`. Separately, `value_loss` compares the current-state prediction with that same target y_t.

Why separate helpers? A target y_t says what the value network should predict; an actor update weight says how to weight a sampled action in the policy loss; a loss is a scalar minimized by an optimizer. `baseline_advantages` and `td_advantages` only prepare actor weights—they do not train the value network. `value_loss` is shared by M2 and M3 because both fit a value prediction to a target; only the target changes. The supplied main program connects these helpers and performs all parameter updates.

For each batch, the program collects eight complete episodes under the unchanged current policy, computes all targets and advantages using pre-update predictions, takes one actor optimization step, then five value-regression steps against the frozen targets. It then discards the batch. This is on-policy actor–critic with one-step *targets*; collecting a batch does not make it off-policy. There is no replay buffer or target network. Do not standardize advantages in the required experiments.

How to read the supplied code

Dimensions and indexing. T is the number of transitions in one episode; B is the number of complete episodes in a training batch; $N = T_1 + \cdots + T_B$ is the total number of transitions in that batch. For example, two episodes of lengths 3 and 5 give $B = 2$ and $N = 8$. Shape `[N]` is a one-dimensional vector, not `[N,1]`; shape `[]` is a single scalar tensor, not a Python float. Training uses floating-point `torch.float32` tensors on the CPU, except for Boolean ending flags and integer

action indices. Preserve the inputs' floating-point type and device in your outputs. Every batch vector uses the same transition order, concatenating complete episodes without padding. Within an episode, entry `rewards[t]` is R_{t+1}, the reward received after taking A_t in S_t.

Quantity	Shape/type	Meaning
Observations	`[N,9]`, float	Eight measurements plus remaining time
Policy logits	`[N,4]`, float	Four action scores per observation
Sampled actions	`[N]`, integer	Action index 0, 1, 2, or 3
Sampled log-probabilities	`[N]`, float	One selected action per transition
State-value predictions	`[N]`, float	One predicted return per state
Ending flags	`[N]`, Boolean	Whether each transition ends the task
Actor/value loss	`[]`, float	One scalar for optimization

Small PyTorch examples. A tensor is an array of numbers. Shape `[N]` means one number for each of N transitions. Arithmetic on equally sized tensors is elementwise; `.sum()` adds the entries and `.mean()` averages them. Use `.detach()` exactly where instructed: it keeps the numbers unchanged but treats them as fixed when computing derivatives.

```
values = torch.tensor([1., 2.], requires_grad=True)
targets = torch.tensor([3., 1.])
fixed_targets = targets.detach()
errors = values - fixed_targets       # [-2., 1.]
loss = (errors ** 2).mean()           # scalar 2.5
```

The supplied training program handles differentiation and parameter updates. Do not detach the entire loss or the predicted values in this example. For the actor loss, detach only the update weights, not the log-probabilities.

Useful Python expressions.

```
torch.empty_like(rewards)       # uninitialized tensor, same shape/type
rewards.new_zeros(())           # scalar zero
range(len(rewards)-1, -1, -1)   # indices in reverse order
```

Assign every entry of a tensor created with `empty_like` before using it. Both tensor-creation helpers preserve the input tensor's data type and device.

1. **`coding/algorithms.py`: the main file you edit.** Implement the six core functions listed below. Each TODO raises `NotImplementedError` until completed. The ECE 517 extension function is separate and is not needed for the core runs.

   - `returns_to_go(rewards, gamma)`: **Inputs:** `rewards` is a floating tensor `[T]` from one episode, including its final reward; `gamma` is a Python float. **Output:** a floating tensor `[T]` whose entry t is G_t. Keep chronological order and the input's type/device. Do not subtract a baseline, change the input in place, or join episodes before computing returns.
   - `policy_loss(log_probs, advantages, discounts, episode_count)`:
     Inputs: `log_probs`, `advantages`, and `discounts` are aligned floating vectors `[N]`. They contain, respectively, the log-probability of each sampled action (not all four logits), its actor

update weight, and γ^t with t reset at each episode. `episode_count` is the Python integer B. **Output:** a differentiable scalar tensor `[]` using the actor equation. Keep gradients through log-probabilities only; detach update weights. Divide by B, not N, and do not apply γ^t twice.

- `baseline_advantages(returns, values)`: **Role:** M2 middle column, $G_t - \hat{v}(S_t)$; its output goes to `policy_loss`, not to the value network as a training target. **Inputs:** two floating vectors `[N]` containing sampled returns G and pre-update predictions $\hat{v}(S)$ for the same states. **Output:** detached floating vector `[N]` containing $G - \hat{v}(S)$, in the same order. Do not average or normalize it.
- `value_loss(values, targets)`: **Role:** the L_v equation in *Algorithms and loss conventions*. It trains the value network in M2 using target G_t and in M3 using target y_t (last column). It is not used in M1 and does not compute the actor loss. **Inputs:** two floating vectors `[N]`: current state-value predictions and fixed training targets (G for the baseline or y for TD). **Output:** scalar tensor `[]` equal to the mean squared error over the N transitions, without a factor 1/2. Keep gradients through `values`, but detach `targets`.
- `td_targets(rewards, next_values, ended, gamma)`: **Role:** M3 last column, y_t. Its output is used twice: as the target in `value_loss` and as the first input to `td_advantages`. It is not yet the actor update weight. **Inputs:** floating vectors `rewards` and `next_values` of shape `[N]`, a Boolean vector `ended` of shape `[N]`, and Python float `gamma`. Each entry describes the same transition; `next_values` contains pre-update $\hat{v}(S_{t+1})$, not $\hat{v}(S_t)$. **Output:** detached floating vector `[N]` of one-step TD targets. Use only the reward where `ended` is true; otherwise include the discounted next-state value.
- `td_advantages(targets, values)`: **Role:** M3 middle column, $y_t - \hat{v}(S_t)$; its output goes to `policy_loss`. Unlike `td_targets`, this helper uses the current-state value to form the actor weight. **Inputs:** floating vectors `[N]` containing TD targets y and pre-update current-state predictions $\hat{v}(S)$ in matching order. **Output:** detached floating vector `[N]` containing $y - \hat{v}(S)$, not a scalar average.

For example, rewards $[1, -2]$, next values $[5, 99]$, ending flags [False, True], and $\gamma = 0.9$ give targets $[5.5, -2]$. Do not use NumPy inside a differentiable loss. Return PyTorch tensors, not Python floats.

2. **`coding/models.py`: supplied network definitions.** The actor and value function are separate networks with two 64-unit tanh hidden layers. The actor maps nine inputs to four logits (unnormalized log-probability scores). `Categorical(logits=...)` handles normalization and action sampling. The value network outputs one scalar per observation. Do not discretize states. No architecture changes are required.

3. **`coding/train.py`: main program with two C4 TODO blocks.** Complete only C4-A and C4-B inside its `update` function. It seeds the generators, creates LunarLander, appends remaining time, collects complete episodes, calls your functions, updates the networks, and evaluates the frozen final policy. The actor learning rate is 3×10^{-4}, value learning rate 10^{-3}, and each network's gradient norm is clipped at 5. The program uses CPU execution with one PyTorch thread and never renders training. It refuses to overwrite an existing output directory; choose a new directory for each run or rerun. A failed run may leave partial files; do not submit it as complete.

4. **Supplied tests.** Numerical checks are in `coding/test_public.py`. Training connections are checked in `coding/test_pipeline.py`. Test settings are in `coding/pytest.ini`. These check

small calculations, output shapes, and gradient detachment. Passing them does not prove that every case or full training behavior is correct. The pipeline tests additionally check the actor/value loss connections and which networks change after an update, using synthetic data. The extension marker separates ECE 517 tests from the core tests.

5. `coding/plot_results.py`: **supplied analysis helper.** It reads completed runs, creates training and evaluation figures and a formatted table image, and also saves the table as a CSV. It rejects duplicate training seeds within one method and comparisons with unequal core hyperparameters. You write the interpretation and assemble `report.pdf`; the helper does not write your report or explanations.

6. `coding/requirements.txt`: **supplied dependency list.** Use a separate Python 3.11 or 3.12 environment. This task needs PyTorch, NumPy, Gymnasium with Box2D, Matplotlib, and pytest. No GPU or MuJoCo installation is required.

Tasks and grading

1. **C1. REINFORCE (20 points). File:** `coding/algorithms.py`. Implement `returns_to_go` [10] and `policy_loss` [10]. Correctly handle the last reward, discount powers, sign, batch scaling, and gradients.

2. **C2. Learned value baseline (15 points). File:** `coding/algorithms.py`. Implement `baseline_advantages` [8] and `value_loss` [7]. The baseline learns complete-episode Monte Carlo targets.

3. **C3. One-step actor–critic (35 points). File:** `coding/algorithms.py`. Implement `td_targets` [20] and `td_advantages` [15]. Use the provided optimization loop and your existing actor/value losses. Do not bootstrap at a task ending or propagate an actor gradient through the critic.

4. **C4. Connect the training pipeline (10 points). File:** `coding/train.py`. Inside `update`, replace each of the two marked `raise NotImplementedError` lines with one assignment:

   - **C4-A: Actor-Step [5].** Set scalar `loss` by calling `alg.policy_loss`. Use the sampled-action `log_probs`, the prepared `advantages` (actor weights), `discounts`, and the number of episodes. These are the inputs to L_π, not a squared prediction error. This step runs for M1, M2, and M3.
   - **C4-B: Critic-Step [5].** Set scalar `critic_loss` by calling `alg.value_loss`. Use fresh `current_values` and the fixed `targets` prepared in Step 1: G for M2 or y for M3. Do not use the actor weights as targets or the old detached predictions as the values to train. This is the squared-error loss L_v.

   Read the surrounding supplied lines: `zero_grad()` clears old gradients, `backward()` computes gradients with autograd, and `step()` lets Adam change the corresponding network weights. Keep these lines unchanged. M1 updates only the actor; M2 and M3 update both networks. The five value-network steps reuse the fixed targets but recompute predictions each time. Run `coding/test_pipeline.py` to check these connections on a small synthetic batch without running a LunarLander training experiment.

5. **C5. Controlled experiments and interpretation (20 points). Run: `coding/train.py`; analyze: `coding/plot_results.py`; write: `report.pdf`.** Run all three methods for seeds 1, 2, and 3 [6]. Present the training plot, evaluation plot, and summary table [6]. Answer the concrete comparison questions in the report instructions below [8]. Use the same 200,000-step requested budget and default hyperparameters for every core run. The final complete batch may exceed that budget by fewer than 8,000 steps; report actual steps and restrict learning-curve comparisons to the shared budget. No fixed return threshold is required. Correct implementation, controlled experiments, and honest analysis matter more than a single favorable run. Any tuned experiments must be additional, labeled separately, with all settings disclosed.

Report and submission workflow

Supplied versus created: We supply source files, tests, dependencies, and plotting tools, but no completed learning functions, trained results, figures, or report. You complete `algorithms.py` and the two marked blocks in `train.py`; the commands generate data and images; you create `report.pdf`. Run every command below from the package root, the directory containing `coding/`.

Step 1: install, implement, and test

In your Python 3.11/3.12 environment, run:

```
python -m pip install -r coding/requirements.txt
python -m pytest coding/test_public.py coding/test_pipeline.py -m "not extension"
```

The second command is expected to fail until the six core functions are complete. The pipeline tests also require the two C4 blocks in `train.py`. If Box2D installation reports a missing SWIG tool, install it with `python -m pip install swig`, then retry the dependency installation. After the tests pass, do a short execution check:

```
python coding/train.py --method actor_critic --steps 1000 \
  --batch-episodes 2 --eval-episodes 2 --seed 1 \
  --output checks/ac
```

This verifies that training and evaluation execute and write files. It is not a learning-performance test and does not replace any required run.

Step 2: generate training and evaluation data

Run all nine commands. Each creates a new run directory containing `training.csv`, `evaluation.csv`, `config.json`, and `agent.pt`. Evaluation samples actions from the final stochastic policy, with frozen parameters and 30 fresh episodes; it is *not* greedy argmax evaluation.

```
python coding/train.py --method reinforce --seed 1 \
  --output results/core/reinforce_seed1
python coding/train.py --method reinforce --seed 2 \
  --output results/core/reinforce_seed2
python coding/train.py --method reinforce --seed 3 \
  --output results/core/reinforce_seed3
```

```
python coding/train.py --method baseline --seed 1 \
  --output results/core/baseline_seed1
python coding/train.py --method baseline --seed 2 \
  --output results/core/baseline_seed2
python coding/train.py --method baseline --seed 3 \
  --output results/core/baseline_seed3

python coding/train.py --method actor_critic --seed 1 \
  --output results/core/ac_seed1
python coding/train.py --method actor_critic --seed 2 \
  --output results/core/ac_seed2
python coding/train.py --method actor_critic --seed 3 \
  --output results/core/ac_seed3
```

How to read `training.csv`. The first row is the header; every later row describes *one completed training episode*, not an optimization step. The row count varies because episode lengths vary.

Column	Meaning
`episode`	Completed training episode number, starting at 1.
`env_steps`	Cumulative training environment interactions through this episode. Evaluation interactions are not included. This is the learning-curve horizontal axis.
`batch`	Batch number, starting at 1. Eight episodes share a batch; learning happens after all eight are collected.
`return`	Sum of original rewards in this episode, *undiscounted*. Use for performance plots; it is not the return-to-go tensor used in the loss.
`length`	Number of actions in the episode, at most 1000. A longer episode does not necessarily mean a better landing.
`seed`	Training seed from the command; constant throughout this file.

How to read `evaluation.csv`. After the header, each of 30 rows describes a fresh episode of the *same* frozen trained agent. There are no optimizer updates during evaluation.

Column	Meaning
`episode`	Evaluation episode number, starting again at 1.
`return`	Undiscounted sum of original rewards; average all 30 to obtain one evaluation mean for this training seed.
`length`	Evaluation episode length; not the reward metric.
`train_seed`	Seed identifying the trained agent being evaluated.
`reset_seed`	Environment reset seed for this evaluation episode: 10^6 `train_seed` + 500000 + `episode` − 1.

For training seed 1, the evaluation reset seeds are 1500000 through 1500029. These are not 30 independent training runs. The action generator is separately seeded with `train_seed`+200000 before evaluation. `config.json` records method, seed, requested budget, hyperparameters, horizon,

and installed package versions. `agent.pt` contains the final trained model. Include it in each submitted run folder alongside `config.json`, `training.csv`, and `evaluation.csv`, so the trained policy can be loaded and evaluated without retraining. Only the final checkpoint for each run is required.

Step 3: generate the figures and summary table

```
python coding/plot_results.py --input results/core \
  --output figures/core
```

This produces four files in `figures/core/`:

- `training.png`: trailing 50-episode mean returns for each seed, interpolated onto a shared environment-step grid, then averaged across seeds. Early points use all available episodes if fewer than 50 exist. Shading is across-seed *sample standard deviation*, not a confidence interval.
- `evaluation.png`: each labeled point is one seed's 30-episode evaluation mean; the black bar is the mean of the three training-seed means.
- `summary.png`: a readable table with method, seed, final 50-episode training mean, evaluation mean, and actual training steps. Aggregate rows give mean ± sample SD across training seeds.
- `summary.csv`: the same table in machine-readable form for verification.

The helper does not extrapolate before a seed's first observation. Submit all three images and embed them legibly in your report. Treat the training smoothing as a display choice; evaluate the final policy using the separate evaluation data.

Step 4: ECE 517 required coding extension E1 (10 points)

In `coding/algorithms.py`, implement `n_step_targets` [6 pts]. Its inputs are one episode's rewards of shape `[T]`, pre-update predictions of shape `[T+1]` (last entry zero), γ, and positive integer n. Return detached targets of shape `[T]` using

$$y_t^{(n)} = \sum_{k=0}^{m-1} \gamma^k R_{t+k+1} + \begin{cases} \gamma^m \hat{v}(S_{t+m}), & t+m < T, \\ 0, & t+m = T, \end{cases} \qquad m = \min(n, T-t).$$

Here T is the episode length and $\hat{v}$ uses the pre-update predictions. For example, take rewards $[1, -1, 3]$, values $[2, 1, 4, 0]$, $\gamma = 0.9$, and $n = 2$. At $t = 0$, use two rewards and then the value at S_2: $y_0 = 1 + 0.9(-1) + 0.9^2(4) = 3.34$. At $t = 1$, the episode ends after two rewards, so $y_1 = -1 + 0.9(3) = 1.7$ with no bootstrap. At $t = 2$, $y_2 = 3$. Thus the output is $[3.34, 1.7, 3]$. With $n = 5$, all remaining rewards fit within the horizon, giving $[2.53, 1.7, 3]$. Do not cross episode boundaries. The supplied main program already handles the optimization loop. Run the extension tests, then $n = 5$ experiments:

```
python -m pytest coding/test_public.py -m extension
python coding/train.py --method actor_critic --n-step 5 --seed 1 \
  --output results/extension/ac5_seed1
python coding/train.py --method actor_critic --n-step 5 --seed 2 \
  --output results/extension/ac5_seed2
python coding/train.py --method actor_critic --n-step 5 --seed 3 \
```

```
  --output results/extension/ac5_seed3
python coding/plot_results.py --input results \
  --output figures/extended
```

Compare the new $n = 5$ results with your existing $n = 1$ runs [4 pts]. Include the additional plots/table. Compare mean evaluation return and variability across seeds for n=1 and n=5; state whether the observed improvement is consistent across seeds. No theoretical bias–variance explanation is required. No new $n = 1$ runs are necessary. ECE 414 students do not need to complete this extension.

Step 5: write and submit your report

Create `report.pdf` with these sections:

1. **Implementation:** briefly explain the six core functions and the two C4 loss calls, the difference between Monte Carlo and TD targets, and the rule that the future value is zero at a task ending.

2. **Experimental setup:** include exact commands, training seeds, package versions from `config.json`, requested/actual budgets, and any deviations.

3. **Core results:** embed `training.png`, `evaluation.png`, and `summary.png`; a carefully typeset version of the same table is also acceptable.

4. **Discussion:** (i) Which method has the highest mean evaluation return? (ii) How much do evaluation results vary across the three training seeds? (iii) Did adding the baseline improve the observed results over REINFORCE alone? (iv) How does one-step actor–critic compare with the other two methods? Cite your plots/table and report mixed or inconclusive results honestly. These runs do not establish a universal ranking or prove why a method performed better.

5. **ECE 517 extension:** include the n=5 comparison if enrolled in ECE 517.

Submit one coding ZIP containing `report.pdf`, completed `coding/`, per-run CSVs, configuration JSONs, and final `agent.pt` checkpoints in `results/`, and generated figures and summary CSVs in `figures/`. The report PDF belongs inside this ZIP, not in a separate upload. Include a final checkpoint for every submitted core run and, for ECE 517, every submitted extension run. No intermediate checkpoints are needed. Exclude `checks/`, caches, and environments. **Submit the written homework separately as a single PDF only.**